

Santiago Córdoba, Gonzalo García y Miguel Gabaldón - 3ºA iMAT

El formato del informe podría estar un poco más cuidado.

Practica 2 – Robots

9,5

Simulación:

TODO: 2.1. Complete the function body with your code (i.e., replace the pass statement). Según PEP 8 debería haber un espacio entre operadores y operandos.

```
angular_vel_left = (v - w*self.TRACK/2) / self.WHEEL_RADIUS
angular_vel_right = (v + w*self.TRACK/2) / self.WHEEL_RADIUS
```

```
if angular_vel_left < self.WHEEL_SPEED_MAX and angular_vel_right <
```

```
self.WHEEL_SPEED_MAX :
```

```
    self._last_right = angular_vel_right
    self._last_left = angular_vel_left
```

¡Estas comprobaciones hay que hacerlas en valor absoluto!

```
self._sim.setJointTargetVelocity(self._motors["left"], self._last_left)
self._sim.setJointTargetVelocity(self._motors["right"], self._last_right)
```

TODO: 2.2. Compute the derivatives of the angular positions to obtain velocities [rad/s].

```
wr = encoders["right"] / self._dt
wl = encoders["left"] / self._dt
```

TODO: 2.3. Solve forward differential kinematics (i.e., calculate z_v and z_w).

```
z_v = (wr + wl) * self.WHEEL_RADIUS / 2
z_w = (wr - wl) * self.WHEEL_RADIUS / self.TRACK
```

```
return z_v, z_w
```

TODO: 2.4. Create the /odometry (Odometry message) and /scan (LaserScan) publishers.

```
qos = QoSProfile(
    reliability=QoSReliabilityPolicy.BEST_EFFORT,
    durability=QoSDurabilityPolicy.VOLATILE,
    history=QoSHistoryPolicy.KEEP_LAST,
    depth=10
)
```

```
self._lidar_publisher = self.create_publisher(
    msg_type=LaserScan,
    topic="scan",
    qos_profile=qos
)
```

```
self._odometry_publisher = self.create_publisher(
    msg_type=Odometry,
    topic="odometry",
    qos_profile=qos
)
```

TODO: 2.5. Complete the function body with your code (i.e., replace the pass statement).

```
msg = Odometry()  
msg.header.stamp = self.get_clock().now().to_msg()  
msg.twist.twist.linear.x = float(z_v)  
msg.twist.twist.angular.z = float(z_w)  
self._odometry_publisher.publish(msg)
```

TODO: 2.6. Complete the function body with your code (i.e., replace the pass statement).

```
msg = LaserScan()  
msg.header.stamp = self.get_clock().now().to_msg()  
msg.angle_increment = math.radians(1.5)  
msg.ranges = z_scan  
self._lidar_publisher.publish(msg)
```

También podríais haber cargado el rango mínimo y máximo del sensor (se puede obtener de self._robot).

TODO: 2.7. Synchronize _compute_commands_callback with /odometry and /scan.

```
self._subscribers: list[message_filters.Subscriber] = []  
qos = QoSProfile(  
    reliability=QoSReliabilityPolicy.BEST_EFFORT,  
    durability=QoSDurabilityPolicy.VOLATILE,  
    history=QoSHistoryPolicy.KEEP_LAST,  
    depth=10  
)
```

Append as many topics as needed

```
self._subscribers.append(  
    message_filters.Subscriber(  
        self,  
        Odometry,  
        "odometry",  
        qos_profile=qos  
    )  
)
```

```
self._subscribers.append(  
    message_filters.Subscriber(  
        self,  
        LaserScan,  
        "scan",  
        qos_profile=qos  
    )  
)
```

```
ts = message_filters.ApproximateTimeSynchronizer(  
    self._subscribers,
```

```
        queue_size=10,
        slop=9
    )

    ts.registerCallback(self._compute_commands_callback)

# TODO: 2.8. Parse the odometry from the Odometry message (i.e., read z_v
and z_w).
z_v: float = odom_msg.twist.twist.linear.x
z_w: float = odom_msg.twist.twist.angular.z

# TODO: 2.9. Parse LiDAR measurements from the LaserScan message (i.e.,
read z_scan).
z_scan: list[float] = scan_msg.ranges

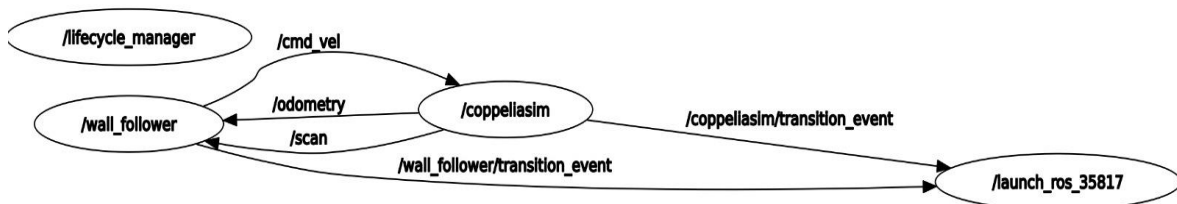
# TODO: 2.10. Create the /cmd_vel velocity commands publisher
(TwistStamped message).
self._cmd_vel_publisher = self.create_publisher(TwistStamped, "cmd_vel",
10)

# TODO: 2.11. Complete the function body with your code (i.e., replace
the pass statement).
new_msg = TwistStamped()
new_msg.header.stamp = self.get_clock().now().to_msg()
new_msg.twist.linear.x = v
new_msg.twist.angular.z = w    Aquí también debería hacer un cast a float() por si acaso.
self._cmd_vel_publisher.publish(new_msg)

# TODO: 2.12. Subscribe to /cmd_vel. Connect it with with
_next_step_callback.
self._cmd_vel_subscriber = self.create_subscription(
    msg_type=TwistStamped,
    topic="cmd_vel",
    callback= self._next_step_callback,
    qos_profile=10
)

# TODO: 2.13. Parse the velocities from the TwistStamped message (i.e.,
read v and w).
v: float = cmd_vel_msg.twist.linear.x
w: float = cmd_vel_msg.twist.angular.z
```

Pregunta #1: Incluya en el informe un diagrama de rqt_graph que permita entender el flujo de información entre los dos nodos que ha programado en la práctica y explíquelo brevemente.



El diagrama muestra un sistema en lazo cerrado entre los nodos /coppeliassim y /wall_follower. El nodo /coppeliassim, publica los datos de sensores a través de los tópicos /scan (los haces del LiDAR) y /odometry (las velocidades del turtlebot). El nodo /wall_follower se suscribe a estos topics, procesa la información del entorno (especialmente los haces del LiDAR) y calcula los comandos de movimiento necesarios para seguir la pared.

Como resultado, publica velocidades lineales y angulares en el tópico /cmd_vel, al que está suscrito /coppeliassim, que ejecuta el movimiento del robot en la simulación.

Wall Follower Algorithm

El algoritmo de seguimiento de pared tiene dos modos de funcionamiento: seguimiento de pared y giro. En condiciones normales, el robot avanza con velocidad lineal constante ($v = 0.15$) y regula su velocidad angular mediante un controlador PD, calculando el error como la diferencia entre la distancia deseada a la pared (SETPOINT) y la distancia medida (izquierda o derecha según qué pared esté siguiendo). Los parámetros del PD se determinaron primero aumentando K_p hasta que el robot siga la pared, pero oscilando considerablemente, y luego añadir el término derivativo K_d e ir subiéndolo hasta que (a ojo) siga mejor la pared. Luego, si detecta un obstáculo frontal por debajo de un umbral, detiene el avance y cambia al modo giro. En este modo, el robot rota sobre sí mismo hacia el lado que estaba siguiendo hasta que la distancia diagonal y frontal indican que puede retomar el seguimiento normal.

TODO: 2.14. Complete the function body with your code (i.e., compute v and w).

```
v = 0.15
```

```
w = 0.0
```

```
dist_front = 0.1 if math.isnan(z_scan[0]) else z_scan[0]
```

```
dist_right = (
```

```
    0.1 if math.isnan(z_scan[3 * len(z_scan) // 4]) else z_scan[3 *
len(z_scan) // 4]
```

```
)
```

```
dist_left = 0.1 if math.isnan(z_scan[len(z_scan) // 4]) else
```

```
z_scan[len(z_scan) // 4]
```

```
dist_wall = self.SETPOINT
```

```
if not self._turn:
```

```
if self._cambio_estado:
    self._cambio_estado = False

    if dist_left < dist_right:
        self._following_wall = -1 # left
    else:
        self._following_wall = 1 # right

dist_wall = dist_left if self._following_wall == -1 else dist_right

error = self.SETPOINT - dist_wall
der_error = (error - self.last_error) / self._dt
w = (self.KP * error + self.KD * der_error) * self._following_wall
self.last_error = error

if dist_front < 0.21:
    v = 0.0
    self._turn = True
    self._cambio_estado = True

elif self._turn:
    v = 0.0

    if self._cambio_estado:
        self._cambio_estado = False
        if dist_left < dist_right:
            self._following_wall = -1 # left
        else:
            self._following_wall = 1 # right

    if self._following_wall == -1:
        dist_diagonal = 0.1 if math.isnan(z_scan[len(z_scan) // 8]) else
z_scan[len(z_scan) // 8]
    elif self._following_wall == 1:
        dist_diagonal = 0.1 if math.isnan(z_scan[7 * len(z_scan) // 8])
else z_scan[7 * len(z_scan) // 8]

    if dist_diagonal > self.DIAGONAL_CONSTANT * self.SETPOINT and
dist_front > 0.6:
        w = 0.0
        self._turn = False
        self._cambio_estado = True
    else:
        w = 0.5 * self._following_wall

return v, w
```

ROBOT REAL

Añadimos un nodo `odometry_node` que deriva dos medidas consecutivas de pose y publica un mensaje `nav_msgs/Odometry` en `/odometry` con `twist.twist.linear.x` y `twist.twist.angular.z`. Esto sigue la Sección 5.1 del enunciado.

```
import rclpy
from rclpy.node import Node
from nav_msgs.msg import Odometry
from transforms3d.euler import quat2euler
import math
import time
from rclpy.qos import QoSProfile, QoSDurabilityPolicy, QoSHistoryPolicy,
QoSReliabilityPolicy
```

```
class OdometryNode(Node):
    def __init__(self):
        super().__init__("odometry_node")
        self.subscription = self.create_subscription(
            msg_type=Odometry,
            topic="odom",
            callback=self.read_odom_msg,
            qos_profile=10
        )

        qos = QoSProfile(
            reliability=QoSReliabilityPolicy.BEST_EFFORT,
            durability=QoSDurabilityPolicy.VOLATILE,
            history=QoSHistoryPolicy.KEEP_LAST,
            depth=10
        )

        self.publisher_ = self.create_publisher(Odometry, "odometry",
        qos_profile=qos)
        self.measurements = []
        self.publish = False
    def read_odom_msg(self, msg):
        x_h = msg.pose.pose.position.x
        y_h = msg.pose.pose.position.y
        quat_w = msg.pose.pose.orientation.w
        quat_x = msg.pose.pose.orientation.x
        quat_y = msg.pose.pose.orientation.y
        quat_z = msg.pose.pose.orientation.z
        _, _, th_h = quat2euler((quat_w, quat_x, quat_y, quat_z))
        th_h %= 2 * math.pi
        th_h_deg = math.degrees(th_h)
        message_time = time.time()
```

3/4

¡Tiene que ir en radianes para que la velocidad angular tenga las unidades adecuadas!

```

if self.publish:
    self.measurements.append([x_h, y_h, th_h_deg, message_time])
    self.odom_approx()
    self.measurements = []
    self.publish = False
else:
    self.measurements.append([x_h, y_h, th_h_deg, message_time])
    self.publish = True

def odom_approx(self):
    """Calculates linear and angular velocity from the measurements
    and publishes an Odometry message."""
    msg = Odometry()
    msg.header.stamp = self.get_clock().now().to_msg()
    linear_velocity = 0.0
    angular_velocity = 0.0
    if len(self.measurements) >= 2:
        x1, y1, th1, t1 = self.measurements[-2]
        x2, y2, th2, t2 = self.measurements[-1]
        dt = t2 - t1
        if dt > 0:
            linear_velocity = math.sqrt((x2 - x1) ** 2 + (y2 - y1) **
2) / dt
            angular_velocity = (th2 - th1) / dt
        msg.twist.twist.linear.x = linear_velocity
        msg.twist.twist.angular.z = angular_velocity
        self.publisher_.publish(msg)

def main(args=None):
    rclpy.init(args=args)
    odometry_node = OdometryNode()

    try:
        rclpy.spin(odometry_node)
    except KeyboardInterrupt:
        pass

    odometry_node.destroy_node()
    rclpy.try_shutdown()

if __name__ == "__main__":
    main()

```

En simulación publicabamos TwistStamped para sincronizar con el simulador. En el robot real, el driver espera geometry_msgs/Twist (sin timestamp). Además, el TurtleBot real

usa criterio dextrógiro, así que mantenemos el signo estándar internamente y solo invertimos ω al publicar (Sección 5.2).

```
# Publisher /cmd_vel cambia según simulation (TwistStamped vs Twist)
if self._simulation:
    self._cmd_vel_publisher = self.create_publisher(TwistStamped,
"cmd_vel", 10)
else:
    self._cmd_vel_publisher = self.create_publisher(Twist, "cmd_vel", 10)

def _publish_velocity_commands(self, v: float, w: float) -> None:
    if self._simulation:
        msg = TwistStamped()
        msg.header.stamp = self.get_clock().now().to_msg()
        msg.twist.linear.x = float(v)
        msg.twist.angular.z = float(w)
    else:
        msg = Twist()
        msg.linear.x = float(v)
        msg.angular.z = float(-w)

    self._cmd_vel_publisher.publish(msg)
```

Los comentarios deberían estar en inglés.

inversión de signo SÓLO al publicar

En simulación el robot “se congela” hasta recibir comandos, pero en el real sigue moviéndose, así que necesitamos ejecutar el control cada Δt fijo con un timer. Por eso guardamos la última odometría y el último scan en atributos con callbacks simples, y creamos un temporizador llama periódicamente al controlador (Sección 5.2).

```
# QoS compatible con LiDAR real
qos = QoSProfile(
    reliability=QoSReliabilityPolicy.BEST_EFFORT,
    durability=QoSDurabilityPolicy.VOLATILE,
    history=QoSHistoryPolicy.KEEP_LAST,
    depth=10
)

if self._simulation:
    # (lo de simulación)
    ...
else:
    # Estado "última medida"
    self._z_v = 0.0
    self._z_w = 0.0
    self._z_scan = [0.22] * 240

    self._odom_subscriber = self.create_subscription(
        Odometry, "odometry", self._process_odom_callback,
        qos_profile=qos
```



```

    )
    self._scan_subscriber = self.create_subscription(
        LaserScan, "scan", self._process_scan_callback, qos_profile=qos
    )

    # Timer: ejecuta el control a periodo fijo cada 0.10s
    self._timer = self.create_timer(0.10,
self._compute_commands_callback)

def _process_odom_callback(self, msg: Odometry):
    self._z_v = msg.twist.twist.linear.x
    self._z_w = msg.twist.twist.angular.z

def _process_scan_callback(self, msg: LaserScan):
    self._z_scan = msg.ranges

def _compute_commands_callback(self, odom_msg=Odometry(),
scan_msg=LaserScan(), pose_msg=PoseStamped()):
    if not pose_msg.localized:
        if self._simulation:
            z_v = odom_msg.twist.twist.linear.x
            z_w = odom_msg.twist.twist.angular.z
            z_scan = scan_msg.ranges
        else:
            # En robot real usamos lo último recibido
            z_v = self._z_v
            z_w = self._z_w
            z_scan = self._z_scan

    v, w = self._wall_follower.compute_commands(z_scan, z_v, z_w)
    self._publish_velocity_commands(v, w)

```

En el robot real el número de haces del LiDAR puede variar y aparecen medidas fuera de rango como nan. Para hacerlo mejor usamos $n = \text{len}(z_scan)$ y seleccionamos índices de los rayos laterales, frontal y diagonales frontales, y añadimos `safe_value()` para reemplazar nan por vecinos o un valor mínimo. También bajamos la velocidad lineal (0.10) y reajustamos umbrales y los parámetros Kp y Kd (1.0 y 5.0 respectivamente). También tuvimos en cuenta de cambiar la pared que seguimos si el error absoluto es mayor que 0.3, cambiando el valor del error, derivada del error, el ultimo valor del error, y el índice de `self._following_wall`.

```

v = 0.10
w = 0.0
n = len(z_scan)
dist_front = self.safe_value(0,z_scan)
dist_right = self.safe_value(3 * n // 4,z_scan)

```

```

dist_left  = self.safe_value(n // 4,z_scan)
dist_wall = self.SETPOINT

if not self._turn:
    if self._cambio_estado:
        self._cambio_estado = False

        if dist_left < dist_right:
            self._following_wall = -1 # left
        else:
            self._following_wall = 1  # right

dist_wall = dist_left if self._following_wall == -1 else
dist_right

self._logger.info(f"{dist_wall}")
error = self.SETPOINT - dist_wall
if abs(error) > 0.3:
    der_error = 0
    error = 0
    self.last_error = None
    self._following_wall *= -1
else:
    der_error = (error - self.last_error) / self._dt if
self.last_error else 0
    self.last_error = error

w = (self.KP * error + self.KD * der_error) *
self._following_wall

if dist_front < 0.25:
    v = 0.0
    self._turn = True
    self._cambio_estado = True

elif self._turn:
    v = 0.0

    if self._cambio_estado:
        self._cambio_estado = False
        if dist_left < dist_right:
            self._following_wall = -1 # left
        else:
            self._following_wall = 1  # right

    if self._following_wall == -1:
        dist_diagonal = self.safe_value(n//8,z_scan)
    elif self._following_wall == 1:
        dist_diagonal = self.safe_value(7*n // 8,z_scan)

```

```
        if dist_diagonal > self.DIAGONAL_CONSTANT * self.SETPOINT and
dist_front > 0.4:
            w = 0.0
            self._turn = False
            self._cambio_estado = True
            # self.last_error = None
        else:
            w = 0.5 * self._following_wall

    return v, w

def safe_value(self,i, z_scan):
    n = len(z_scan)
    if not math.isnan(z_scan[i]):
        return z_scan[i]

    left  = z_scan[(i - 1) % n]
    right = z_scan[(i + 1) % n]

    left_valid  = not math.isnan(left)
    right_valid = not math.isnan(right)

    if left_valid and right_valid:
        return 0.5 * (left + right)
    elif left_valid:
        return left
    elif right_valid:
        return right
    else:
        return 0.1
```